

- a. Code complexity;
 - b. Application binary—the key consideration is to look at the mobile device storage capacity;
 - c. Side effects owing to code conflicts, as more or less all MAM SDK codes will be leveraging similar events within the application;
 - d. Performance degradation of the application.
- Maintenance overheads with constant upgrades for the latest MAM SDK library updates.
 - Unavailability of the SDK can become a bottleneck. While working on WFM, initially (Q1, 2015) we integrated an iOS variant with MI's App connect library. For Android, the MI MAM SDK/library wasn't available.
2. **App wrapping methodology:** In this methodology, the already compiled and packaged mobile app is wrapped with MAM (EMM) vendor dynamic libraries, and this is called app wrapping. MAM libraries are layered over the already built mobile application binary and then the complete set is recompiled, repackaged and resigned with the EMM app signing certificate to generate a new MAM capable mobile app binary. Post wrapping, standard system calls from the original mobile app are routed through the MAM API library to ensure that the calls are secured and managed. This methodology does not require any development work, that is, no code change is required to hook the MAM SDK. There are several EMM solutions in the market and, with this approach, the mobile application needs to be wrapped with the MAM SDK of all EMMs. The following are the pros and cons of this approach.

Pros

- No development/code change is required.
- Public mobile apps from third party ISVs/developers can be covered as well.

Cons

- Wrapping public apps from third party ISVs/developers or even private apps isn't right and is not recommended. It violates app terms and copyright rules.
- Not a reliable methodology, as it creates a lot of issues and side effects. For WFM, initially (Q1, 2015) we used it for both Android and iOS variants. Post wrapping (with old wrapping engine versions from MI and AW), the app used to get stuck at the landing page with a blank blue screen. Later, after a couple of months with newer wrapping engine versions, the app was able to move ahead from the landing page but used to crash randomly in different modules. On detailed research on Android variants, we found that the MI wrapped library had issues with Implicit Intent handling within the *resolveActivity* method of the *PackageManager* class from the Android OS.
- It can interfere and obstruct certain functionalities of the app. For WFM Android wrapped with MI, we found that the MI MAM libraries were not allowing the app to fetch GZip data from the server, and there wasn't any way to configure and allow it. This became a big bottleneck and we had to drop

this approach eventually.

- Wrapped apps may not support full blown MAM feature sets and will not be able to provide fine grained control like in the SDK approach.
 - For WFM Android, post wrapping with MI and AW, we ran into the blocker issue of reaching the 64k method count limit of Dalvik. Android apps run on Dalvik VM (DVM).
 - Wrapping with multiple EMM (MAM) libraries created conflicts.
 - For WFM, we had to add a different EMM specific code to receive dynamic app configuration from EMM.
- Old methodologies could achieve varying levels of containerisation and MAM for small/medium mobile applications but had several downsides as mentioned above. Moreover, the rapidly changing market introduced several methodologies and thus fragmented the market. It created a lot of confusion and chaos amongst IT admins, product owners and app developers to identify the right way to achieve containerisation and MAM.

OS containerisation – standard and recommended EMM integration methods

In the past few years, Apple and Google realised the increasing use of personal mobile devices for work and the need for standardisation. So they took the initiative to bake in containerisation and MAM capabilities right into the mobile OS, that is, iOS and Android (AndroidforWork or AFW). Mobile OS native containerisation can be thought of as a new, standard, universal approach. I will term it as OS containerisation for the rest of this article.

The AppConfig community (a group of EMM providers, ISVs/developers and enterprises) has been formed to standardise and streamline the OS containerisation and integration process. It has come out with an EMM independent methodology to leverage OS containerisation features. As a result, many of the management and security features are automatically taken care of by the OS and will not require any development. For a few features, like app configuration, minimal but standard OS code changes can be made so as to receive dynamic app configuration values from any EMM. With OS containerisation, managed mobile applications can be governed by any app configuration member EMM without any EMM specific code. The following are the pros and cons of this approach.

Pros

- Reliable approach, as it is backed by mobile OS vendors (Apple/Google), EMMs, enterprises, ISVs/developers.
- No conflict, nor code redundancy via a single unified, standard mobile OS infrastructure, code for containerisation and MAM.
- No development/code changes are needed.
- Internal (system) apps and public mobile apps from third party ISVs/developers can be covered under this.
- No EMM vendor lock-in, as IT admin can change the existing EMM with another app configuration member EMM, and it will work seamlessly.
- No 64k method count problem on Android.

Continued to page 103...